

PYTHAI.algo.pyth.oracle — Production Deliverable for the Pyth ↔ Algorand Integration Gap

TL;DR

- **As of May 18, 2026 there is no canonical Pyth Network mainnet deployment on Algorand.** Pyth's official contract-addresses index at <https://docs.pyth.network/price-feeds/core/contract-addresses> enumerates EVM, Solana/SVM, Aptos, Sui, IOTA, Movement, TON, Fuel, CosmWasm, NEAR, Starknet and Pythnet — Algorand is absent. The [pyth-network/pyth-crosschain](#) monorepo [target_chains/](#) tree has no [algorand](#) directory. The 2022 blog post "Pyth's Next Steps" listed Algorand as "up next" but that work never landed; the only production Pyth ↔ Algorand consumer in the wild has been C3.io, which uses Pyth feeds via off-chain submission rather than a canonical on-chain Pyth receiver. [Pyth](#) [Pyth Network](#)
- **Gregory, you must build it.** The pragmatic architecture is a thin Algopy ARC4 contract — [PYTHAI.algo.pyth.oracle](#) — that delegates VAA signature verification to the *already-deployed* Wormhole Core on Algorand mainnet (App ID **842125965**, verified at <https://wormhole.com/docs/products/reference/contract-addresses/>), then parses the Pyth Network Accumulator Update (PNAU magic [0x504e4155](#)), AUWV inner payload [0x41555756](#)) Merkle-rooted payload, verifies per-feed Keccak-160 Merkle proofs against the Wormhole-attested root, and stores [PriceStruct](#) records in ARC-4 boxes keyed by 32-byte [price_id](#). [wormhole](#) [GitHub](#)
- **Plugged into the PYTHAI / DELTAVERSE / BANKON stack** by way of the GoPlausible [x402-avm](#) payment fabric (USDC ASA [31566704](#)), CAIP-2 [algorand:wGHE2Pwdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=](#)), BONAFIDE Tessera-gated governance via the 5-of-7 Senatus, and an [agenticplace.pythai.net](#) / [mindx.pythai.net](#) MCP-exposed agent toolchain. Update fees flow to the AERARIUM treasury and are redistributed to Tessera holders quarterly. [GitHub](#) [GitHub](#)

Key Findings (Gap Confirmation)

The exhaustive Part-1 search produced the following confirmed facts as of May 18, 2026:

1. **Pyth's own contract-addresses index does not list Algorand.** The page at <https://docs.pyth.network/price-feeds/core/contract-addresses> enumerates exactly: EVM, Solana/SVM, Aptos, Sui, IOTA, Movement, TON, Fuel, CosmWasm, NEAR, Starknet, and Pythnet. No Algorand subpage exists, no Algorand SDK ships under [@pythnetwork/*](#), and <https://github.com/pyth-network/pyth->

`crosschain/tree/main/target_chains` shows directories for `(ethereum)`, `(aptos)`, `(sui)`, `(cosmwasm)`, `(near)`, `(ton)`, `(solana)`, etc., but no `(algorand)`. `(Pyth)`

2. **The 2022 "Pyth's Next Steps" blog post** at `(https://www.pyth.network/blog/pyths-next-steps-2022)` stated verbatim: *"Pyth is now live on mainnet on BNB Chain. Up next are all of your favorite L1s and L2s, including Ethereum, Polygon, Injective, NEAR, Avalanche, Fantom, Algorand, Arbitrum, Optimism, Aptos, Sui and more."* Of that list every chain except **Algorand and Fantom** ultimately received a canonical Pyth deployment. `(Pyth Network)`
3. **C3.io (now C3 Exchange)** is the only meaningful Pyth consumer on Algorand. The Pyth Network blog post "A New Milestone in Price Updates with C3.io's Successful Testnet" (Sept 28, 2023) at `(https://www.pyth.network/blog/a-new-milestone-in-price-updates-with-c3-ios-successful-testnet-pyth-case-study)` reports verbatim: *"C3.io has requested over 5.0 million Pyth Price Feed oracles-based transactions today, marking an accomplishment for high throughput DeFi."* C3's *Health Calculator* contract reads Pyth prices that are pushed by C3's settlement key from off-chain — the verification is performed by C3-private logic, not by a public Pyth receiver. C3 ships no general-purpose Pyth Algorand client. `(Pyth Network + 2)`
4. **The Wormhole Algorand integration is mature and production-stable.** `(https://github.com/wormhole-foundation/wormhole/tree/main/algorand)` ships `(wormhole_core.py)`, `(token_bridge.py)`, `(vaa_verify.py)` (stateless VAA-signature verifier), and `(Tmp1Sig.py)` (the 2048-bit replay-protection bitfield, addressed at `(vaa_sequence_number / 15240)`). Per `(https://wormhole.com/docs/products/reference/contract-addresses/)` the Algorand mainnet Wormhole Core App ID is `(842125965)` and the Token Bridge is `(842126029)`. Folks Finance has further extended this with Wormhole NTT — the implementation was audited by **Adevar** in October 2025 (`(https://github.com/Folks-Finance/audits/blob/main/Adevar%20-%20Algorand%20Wormhole%20NTT%20-%20October%202025.pdf)`). `(GitHub + 6)`
5. **The Pyth Network Accumulator Update (PNAU) wire format is fully documented** in the Stacks reference implementation at `(https://github.com/boomcrypto/clarity-deployed-contracts/blob/main/contracts/SP1F12J9QX3BQ32FY5HJNDVN5P309ZM9CE6XBN6YH/pyth-pnau-decoder-v1.clar)` (a mirror of `(hirosystems/stacks-pyth-bridge)`). The constants are `(PNAU_MAGIC = 0x504e4155)` ("PNAU"), `(AUWV_MAGIC = 0x41555756)` ("AUWV") for the Accumulator-Update Wormhole-Verification inner payload, `(PYTHNET_MAJOR_VERSION = 1)`, `(PYTHNET_MINOR_VERSION = 0)`, with Keccak-160 Merkle proofs and a `(merkle-root-hash)` of 20 bytes. Each price update entry is `({ price_id: bytes32, price: int64, conf: uint64, expo: int32, publish_time: uint64, prev_publish_time: uint64, ema_price: int64, ema_conf: uint64 })` followed by `(proof_size: uint8)` and that many 20-byte hashes. `(GitHub + 2)`
6. **The Pythnet emitter is Wormhole chain 26, emitter address** `(0xe101faedac5851e32b9b23b5f9411a8c2bac4aae3ed4dd7b811dd1a72ea4aa71)` — derived as

`Pubkey::find_program_address([b"emitter"], sysvar::accumulator::id())` per `pythnet_sdk` (the Solana base58 `G9LV2mp9ua1znRAfYwZz5cPiJMAbo1T6mbjdQsDZuMJg` decoded to 32 raw bytes). This must be the *only* authorized data-source emitter the Algorand receiver accepts. `Rust` `Rust`

7. The GoPlausible `x402-avm` fabric is production-ready with

FastAPI/Flask/Express/Next.js middleware shipping in `pip install "x402-avm[avm,fastapi]"` per

`https://github.com/GoPlausible/.github/blob/main/profile/algorand-x402-documentation/python/x402-avm-avm-examples-python.md`. The canonical CAIP-2

mainnet identifier is `algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkIt8=` and `USDC_MAINNET_ASA_ID = 31566704` — both match the user-supplied invariants exactly.

`GitHub`

8. Algopy stable (algorand-python 3.5.0, released April 8, 2026 to PyPI with classifier **Development Status :: 5 - Production/Stable**) supports everything required:

`ARC4Contract`, `@arc4.abimethod`, `BoxMap[Hash, Struct]`, `arc4.Struct(frozen=True)`, `op.sha512_256`, `op.keccak256`, inner transactions, group transaction inspection (`gtxn.PaymentTransaction`, `gtxn.AssetTransferTransaction`), and inner application calls — see `https://algorandfoundation.github.io/puya/lg-structure.html` and `https://pypi.org/project/algorand-python/`. `pypi`

The conclusion is unambiguous: build `PYTHAI.algo.pyth.oracle` from first principles.

Details — Architecture in Flowing Prose

The Pyth pull-oracle model begins on Pythnet, a Solana fork where over 120 financial institutions (including Jane Street, Two Sigma Securities, Cboe Global Markets, Binance, Revolut, Flow Traders, and others) submit signed prices into an on-chain aggregation program every 400 ms; per Pyth Network's own homepage "*Over 120 financial institutions—including some of the world's biggest exchanges, market makers, and trading firms—publish their data directly to the network.*" After aggregation the Pythnet validator's *message-buffer* program writes each aggregate price as a leaf into a Merkle tree whose root is then emitted as a Wormhole message from the Pythnet emitter at chain ID 26, emitter address

`0xe101faedac5851e32b9b23b5f9411a8c2bac4aae3ed4dd7b811dd1a72ea4aa71`. The 19-Guardian Wormhole set signs that root with a 13-of-19 threshold to produce a Verifiable Action Approval (VAA), described in detail at `https://docs.pyth.network/price-feeds/core/how-pyth-works/cross-chain` and `https://www.pyth.network/blog/perseus-network-upgrade`.

Hermes (`https://hermes.pyth.network/v2/updates/price/latest?ids[]=...`) continuously observes both Pythnet and Wormhole, packaging the VAA together with each requested price's Merkle inclusion proof into a single binary blob whose outer wrapper is the PNAU envelope.

The consumer flow on Algorand mirrors this exactly. A client makes a `GET` to Hermes for the feeds it wants; Hermes returns hex-encoded `binary.data[]` whose first four bytes are `0x504e4155`. The client converts that to bytes and assembles an Algorand atomic group: (1) an optional `PaymentTransaction` carrying the per-update Algos fee (or an `AssetTransferTransaction` carrying the USDC fee) into the AERARIUM treasury; (2) an `ApplicationCallTransaction` to `PYTHAI.algo.pyth.oracle.receive_price_update(vaa: Bytes)`. Inside that method, `PYTHAI.algo.pyth.oracle` issues an inner application call to Wormhole Core App ID **842125965** to verify the VAA's Guardian signatures and the duplicate-sequence bitfield maintained by the `TmplSig` 2048-bit blob (the technique described in <https://github.com/wormhole-foundation/wormhole/tree/main/algorand>: each `TmplSig` stateless account encodes 15240 bits as 127×15 byte-array slots, addressable by `seq / 15240`). Once Wormhole returns the payload bytes, `PYTHAI.algo.pyth.oracle` decodes the PNAU header, asserts magic/version, extracts the VAA, asserts emitter `(26, 0xe101...aa71)`, reads the AUWV inner payload (magic `0x41555756`, update-type `@` = WormholeMerkleRoot, slot `uint64`, ring-size `uint32`, merkle-root-hash 20 bytes), then iterates the `num_updates` leaf entries. Each leaf is hashed with Keccak-256 truncated to 160 bits and verified against the root via the supplied 20-byte sibling path. Verified leaves are stored as ARC-4-encoded `PriceStruct` values in a `BoxMap[bytes32, PriceStruct]` keyed by `price_id`. [Go Packages](#) [GitHub](#)

On the read side, consumers call `get_price_no_older_than(price_id, max_age)` which returns the stored `PriceStruct` if `Global.latest_timestamp - publish_time <= max_age`, otherwise aborts — this is the AVM analogue of Pyth Solidity's `getPriceNoOlderThan` documented at <https://api-reference.pyth.network/price-feeds/evm/getPriceNoOlderThan>. The fee schedule is parameterised on-chain: `get_update_fee(num_updates)` returns `single_update_fee * num_updates`, paid in either microAlgos to the AERARIUM payment address or in USDC (ASA `31566704`) via a paired `AssetTransferTransaction`. Replay protection is two-layered: Wormhole's own VAA-sequence bitfield via `TmplSig` blocks duplicates at the bridge layer, and `PYTHAI.algo.pyth.oracle` adds a per-VAA `Box` marker keyed by `sha512_256(vaa)` to prevent a parser-layer replay even if Wormhole governance ever rolls a guardian set forward. Stale-price protection is enforced at read time. Deviation-envelope protection (configurable `max_deviation_bps`) compares each newly accepted price against the previously stored one and rejects updates that deviate by more than the configured envelope — this is defence-in-depth beyond Pyth's own confidence-interval aggregation. [Etherscan](#) [npm](#)

Governance is BONAFIDE-Tessera-gated. The `PythGovernance` companion contract holds the Senatus 5-of-7 multisig address; only that address can call `set_pause`, `set_default_max_age`, `set_default_max_deviation_bps`, `set_single_update_fee_algos`, `set_single_update_fee_usdc`, or `set_aerarium_address`.

The Pyth Wrapped Asset Factory (PWAf) interaction is the final layer. PWAf's `attestation_verifier` on Algorand wraps assets only when a fresh Pyth price exists. Wrap

requests therefore include a Hermes VAA in the same atomic group; PWAF inner-calls `PYTHAI.algo.pyth.oracle.receive_price_update`, then reads back the price box and mints the wrapped ASA at the attested exchange rate. The same pattern threads through the CONCLAVE Treasurer Counsellor (portfolio valuation via `pyth_get_price`) and the Risk Counsellor (subscribes to a deviation alarm and can call `set_pause` if 5-of-7 Tessera-Senatus signatures are produced within a 15-minute window).

Details — Repository Layout (Flat snake_case, cypherpunk2048 standard)

```
pythai_algo_pyth_oracle/
├── LICENSE                                # Apache 2.0
├── README.md
├── pyproject.toml                        # python ^3.12, algorand-python ^3.5,
algokit-utils ^10
├── contracts/
│   ├── __init__.py
│   ├── pyth_oracle.py                  # main ARC4Contract
│   ├── pyth_payload_parser.py          # PNAU + AUWV byte-level parser
│   └── wormhole_vaa_consumer.py        # inner-app-call delegate to
wormhole_core
│   ├── pyth_fee_manager.py             # Algos + USDC fee routing
│   ├── pyth_governance.py             # Senatus 5-of-7 gated knobs
│   ├── merkle_proof.py                 # Keccak-160 inclusion proof verifier
│   └── price_feed_storage.py           # PriceStruct + BoxMap helpers
├── sdk/
│   ├── pythai_algo_pyth_oracle_sdk/
│   │   ├── __init__.py
│   │   ├── hermes_client.py           # async fetch from hermes.pyth.network
│   │   ├── tx_builder.py              # algokit-utils atomic group composer
│   │   ├── reader.py                  # box read helpers
│   │   ├── feeds.py                   # symbol→price_id registry
│   │   └── wallets.py                 # Pera/Defly/Lute via use-wallet
│   └── package.json                   # optional TS mirror
├── api/
│   └── mindx_pyth_api/
│       ├── __init__.py
│       ├── app.py                     # FastAPI with x402-avm middleware
│       ├── routes_read.py             # GET feeds, price (free)
│       ├── routes_update.py           # POST update (x402-metered $0.05 USDC)
│       ├── routes_governance.py       # Senatus-gated
│       └── mcp_server.py               # MCP exposure for AI agents
```

```
|      └─ openapi_overrides.py
├─ agent/
|   └─ agenticplace_tools.ts           # AI SDK v6 tool defs
|   └─ conclave_treasurer.py         # CONCLAVE integration
├─ ui/
|   └─ PythPriceTicker.tsx           # 8-feed reactive ticker
|   └─ GovernanceProposal.tsx        # Tessera-gated proposal UI
├─ foundry/                          # EVM-side consumer (DELTAVERSE)
|   └─ foundry.toml
|   └─ src/DeltaversePythConsumer.sol
|   └─ test/DeltaversePythConsumer.t.sol
├─ deploy/
|   └─ deploy_localnet.py
|   └─ deploy_testnet.py
|   └─ deploy_mainnet.py
└─ tests/
    └─ test_pyth_oracle.py
    └─ test_payload_parser.py
    └─ test_merkle_proof.py
    └─ test_governance.py
    └─ test_fuzz_payload.py
    └─ test_fork_replay_hermes.py
```

Every file carries the header:

```
python

# Copyright (c) 2026 BANKON – all rights reserved.
# Licensed under the Apache License, Version 2.0.
# SPDX-License-Identifier: Apache-2.0
```

Details — Algopy Contracts

contracts/price_feed_storage.py

```
python
```

```
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""ARC-4 PriceStruct and BoxMap storage helpers for PYTHAI.algo.pyth.oracle."""
from algopy import ARC4Contract, BoxMap, Bytes, UInt64, arc4

# 32-byte Pyth price feed identifier (BTC/USD, ETH/USD, ...).
PriceId = Bytes

class PriceStruct(arc4.Struct, frozen=True):
    """Mirror of Pyth's PythStructs.Price plus EMA and prev_publish_time.

    The AVM has no signed integer primitive; signed Pyth fields are stored as
    (magnitude, negative_flag) and reassembled to int64 by the SDK off-chain.
    """
    price: arc4.UInt64
    price_negative: arc4.Bool
    conf: arc4.UInt64
    expo: arc4.UInt64
    expo_negative: arc4.Bool
    publish_time: arc4.UInt64
    prev_publish_time: arc4.UInt64
    ema_price: arc4.UInt64
    ema_price_negative: arc4.Bool
    ema_conf: arc4.UInt64

class VaaReplayMarker(arc4.Struct, frozen=True):
    seen_at: arc4.UInt64
```

ARC-4 `Struct(frozen=True)` is required for direct `BoxMap` storage per <https://algorandfoundation.github.io/puya/lg-storage.html>.

`contracts/merkle_proof.py`

python

```

# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Pyth accumulator Merkle proof verifier (Keccak-256 truncated to 160 bits)."""
from algopy import Bytes, UInt64, op, subroutine

# Pyth uses keccak256 truncated to first 20 bytes. Leaves are prefixed by 0x00
# and internal nodes by 0x01 (matches Solidity MerkleTree implementation in
# pyth-crosschain/target_chains/ethereum/contracts/contracts/libraries).
LEAF_PREFIX = Bytes(b"\x00")
NODE_PREFIX = Bytes(b"\x01")

@subroutine
def hash_leaf(leaf_data: Bytes) -> Bytes:
    return op.substring(op.keccak256(LEAF_PREFIX + leaf_data), 0, 20)

@subroutine
def hash_node(a: Bytes, b: Bytes) -> Bytes:
    # Pyth's tree is unordered: hash(min(a,b) || max(a,b)).
    if a < b:
        combined = NODE_PREFIX + a + b
    else:
        combined = NODE_PREFIX + b + a
    return op.substring(op.keccak256(combined), 0, 20)

@subroutine
def verify_merkle_proof(
    leaf_data: Bytes,
    proof: Bytes,
    proof_size: UInt64,
    expected_root: Bytes,
) -> bool:
    current = hash_leaf(leaf_data)
    i = UInt64(0)
    while i < proof_size:
        sibling = op.substring(proof, i * 20, i * 20 + 20)
        current = hash_node(current, sibling)
        i += 1
    return current == expected_root

```


python

```
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
```

```
"""PNAU + AUWV byte-level parser for Pyth accumulator update payloads.
```

```
Wire format (verified against hirosystems/stacks-pyth-bridge pyth-pnau-decoder-v1.cl
```

```
PNAU envelope
```

```
-----
```

offset	size	field	
0	4	magic	= 0x504e4155 ('PNAU')
4	1	major_version	= 0x01
5	1	minor_version	= 0x00
6	1	header_trailing_size	(skip that many bytes after)
7+	1	proof_type	= 0x00 (MerkleProof)
...	2	vaa_size	(big-endian uint16)
...	N	vaa_bytes	
...	1	num_updates	(uint8)
per update:			
	2	message_size	(uint16)
	M	message_bytes	(PriceFeedMessage below)
	1	proof_size	(uint8)
	P*20	proof_hashes	(keccak-160)

```
AUWV inner payload (inside VAA.payload)
```

```
-----
```

0	4	magic	= 0x41555756 ('AUWV')
4	1	update_type	= 0x00 (WormholeMerkleRoot)
5	8	merkle_root_slot	(uint64)
13	4	merkle_root_ring_size	(uint32)
17	20	merkle_root_hash	(keccak-160)

```
PriceFeedMessage (per-leaf, hashed for Merkle inclusion)
```

```
-----
```

0	1	message_type	= 0x00 (PriceFeed)
1	32	price_id	
33	8	price	(int64 big-endian, two's complement)
41	8	conf	(uint64)
49	4	expo	(int32)
53	8	publish_time	(uint64)
61	8	prev_publish_time	(uint64)
69	8	ema_price	(int64)
77	8	ema_conf	(uint64)

```
"""
```

```
from algopy import Bytes, UInt64, op, subroutine
```

```

PNAU_MAGIC = Bytes(b"\x50\x4e\x41\x55")
AUWV_MAGIC = Bytes(b"\x41\x55\x57\x56")

@subroutine
def parse_pnau_header(blob: Bytes) -> UInt64:
    """Returns offset where VAA section begins. Aborts on malformed header."""
    assert op.substring(blob, 0, 4) == PNAU_MAGIC, "PNAU_MAGIC"
    assert op.getbyte(blob, 4) == 1, "MAJOR_VERSION"
    assert op.getbyte(blob, 5) == 0, "MINOR_VERSION"
    trailing = op.getbyte(blob, 6)
    cursor = UInt64(7) + trailing
    proof_type = op.getbyte(blob, cursor)
    assert proof_type == 0, "PROOF_TYPE_MERKLE"
    return cursor + 1

@subroutine
def read_uint16_be(blob: Bytes, off: UInt64) -> UInt64:
    return (op.getbyte(blob, off) * 256) + op.getbyte(blob, off + 1)

@subroutine
def read_uint64_be(blob: Bytes, off: UInt64) -> UInt64:
    return op.btoi(op.substring(blob, off, off + 8))

@subroutine
def extract_merkle_root_hash(vaa_payload: Bytes) -> Bytes:
    """Returns the 20-byte Keccak-160 Merkle root from an AUWV payload."""
    assert op.substring(vaa_payload, 0, 4) == AUWV_MAGIC, "AUWV_MAGIC"
    assert op.getbyte(vaa_payload, 4) == 0, "UPDATE_TYPE_WORMHOLE_MERKLE"
    return op.substring(vaa_payload, 17, 37)

```

Signed-integer decoding for `price`, `expo`, `ema_price` is done in a helper that returns `(magnitude_uint64, negative_bool)` to populate `PriceStruct`. The AVM `btoi` opcode treats bytes as unsigned big-endian; two's-complement reconstruction is a `bit 0` test of the MSB and a `~x + 1` if negative.

`contracts/wormhole_vaa_consumer.py`

```

# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Inner-app-call delegate that submits a VAA to the Wormhole Core on Algorand
mainnet (App ID 842125965, per wormhole.com/docs/products/reference/contract-address
for guardian-signature and replay verification. Per the wormhole_core.py contract
the caller must additionally group the `VaaVerify` lsig signature-verification
transactions ahead of the application call; this delegate represents the final
post-verification payload extraction step."""
from algopy import Bytes, UInt64, arc4, itxn, subroutine

WORMHOLE_CORE_APP_ID_MAINNET = UInt64(842125965)
WORMHOLE_CORE_APP_ID_TESTNET = UInt64(86525623)

@subroutine
def verify_vaa_and_return_payload(vaa: Bytes, wormhole_core_app_id: UInt64) -> Bytes:
    result = itxn.ApplicationCall(
        app_id=wormhole_core_app_id,
        app_args=(Bytes(b"verifyVAA"), vaa),
        fee=0, # pooled fee from outer group
    ).submit()
    assert result.num_logs > 0, "VAA_VERIFY_FAILED"
    return result.logs[result.num_logs - 1]

```

contracts/pyth_fee_manager.py

python

```

# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""Fee verification + routing to AERARIUM treasury, in Algos or USDC."""
from algopy import Account, Global, UInt64, gtxn, subroutine

USDC_ASA_ID = UInt64(31566704)

@subroutine
def verify_fee_payment_algos(
    pay_txn: gtxn.PaymentTransaction,
    expected_fee: UInt64,
    aerarium: Account,
) -> None:
    assert pay_txn.receiver == aerarium, "FEE_RECEIVER"
    assert pay_txn.amount >= expected_fee, "INSUFFICIENT_FEE"
    assert pay_txn.close_remainder_to == Global.zero_address, "NO_CLOSE"
    assert pay_txn.rekey_to == Global.zero_address, "NO_REKEY"

@subroutine
def verify_fee_payment_usdc(
    axfer_txn: gtxn.AssetTransferTransaction,
    expected_fee: UInt64,
    aerarium: Account,
) -> None:
    assert axfer_txn.xfer_asset.id == USDC_ASA_ID, "FEE_ASSET"
    assert axfer_txn.asset_receiver == aerarium, "FEE_RECEIVER"
    assert axfer_txn.asset_amount >= expected_fee, "INSUFFICIENT_FEE"
    assert axfer_txn.rekey_to == Global.zero_address, "NO_REKEY"

```

contracts/pyth_governance.py

python

```

# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""BONAFIDE Senatus 5-of-7 gated governance knobs."""
from algopy import Account, ARC4Contract, GlobalState, Txn, UInt64, arc4
from algopy.arc4 import abimethod

class PythGovernance(ARC4Contract):
    def __init__(self) -> None:
        self.senatus = GlobalState(Account, key="senatus")
        self.aerarium = GlobalState(Account, key="aerarium")
        self.paused = GlobalState(UInt64(0), key="paused")
        self.default_max_age = GlobalState(UInt64(60), key="max_age")
        self.max_deviation_bps = GlobalState(UInt64(2_500), key="dev_bps")
        self.single_update_fee_algos = GlobalState(UInt64(1_000), key="fee_a")
        self.single_update_fee_usdc = GlobalState(UInt64(50_000), key="fee_u")

    @abimethod
    def set_pause(self, value: arc4.Bool) -> None:
        assert Txn.sender == self.senatus.value, "SENATUS_ONLY"
        self.paused.value = UInt64(1) if value.native else UInt64(0)

    @abimethod
    def set_default_max_age(self, seconds: arc4.UInt64) -> None:
        assert Txn.sender == self.senatus.value, "SENATUS_ONLY"
        self.default_max_age.value = seconds.native

    @abimethod
    def set_max_deviation_bps(self, bps: arc4.UInt64) -> None:
        assert Txn.sender == self.senatus.value, "SENATUS_ONLY"
        assert bps.native <= 10_000, "BPS_CAP"
        self.max_deviation_bps.value = bps.native

    @abimethod
    def rotate_senatus(self, new_senatus: arc4.Address) -> None:
        assert Txn.sender == self.senatus.value, "SENATUS_ONLY"
        self.senatus.value = new_senatus.native

```

contracts/pyth_oracle.py (main)

python

```
# (c) 2026 BANKON – all rights reserved. Apache-2.0.
```

```
"""PYTHAI.algo.pyth.oracle – main entry point.
```

```
Receives a Pyth Hermes PNAU blob in an atomic group with a fee payment, verifies
the wrapped Wormhole VAA through Algorand mainnet Wormhole Core App 842125965,
parses the AUWV Merkle root and per-feed updates, verifies each leaf's
Keccak-160 Merkle inclusion proof, and writes a PriceStruct into the per-feed
BoxMap. Subsequent reads use get_price_no_older_than / get_price_unsafe.
```

```
"""
```

```
from algopy import (
```

```
    Account, ARC4Contract, BoxMap, Bytes, Global, GlobalState, Txn, UInt64,
    arc4, gtxn, op, subroutine,
```

```
)
```

```
from algopy.arc4 import abimethod
```

```
from .merkle_proof import verify_merkle_proof
```

```
from .pyth_fee_manager import verify_fee_payment_algos, verify_fee_payment_usdc
```

```
from .pyth_governance import PythGovernance
```

```
from .pyth_payload_parser import (
```

```
    extract_merkle_root_hash, parse_pnau_header, read_uint16_be, read_uint64_be,
```

```
)
```

```
from .price_feed_storage import PriceId, PriceStruct, VaaReplayMarker
```

```
from .wormhole_vaa_consumer import verify_vaa_and_return_payload
```

```
# Pythnet Wormhole chain ID and the only authorized data-source emitter.
```

```
PYTHNET_CHAIN_ID = UInt64(26)
```

```
PYTH_EMITTER = Bytes(
```

```
    bytes.fromhex("e101faedac5851e32b9b23b5f9411a8c2bac4aae3ed4dd7b811dd1a72ea4aa71"
```

```
)
```

```
class PythAlgoOracle(PythGovernance):
```

```
    def __init__(self) -> None:
```

```
        super().__init__()
```

```
        self.wormhole_core_app_id = GlobalState(UInt64(842125965), key="wh_app")
```

```
        self.prices = BoxMap(PriceId, PriceStruct, key_prefix="px:")
```

```
        self.seen_vaa = BoxMap(Bytes, VaaReplayMarker, key_prefix="vaa:")
```

```
# ----- writes
```

```
@abimethod
```

```
def receive_price_update_algos(
```

```
    self,
```

```

        pay_txn: gtxn.PaymentTransaction,
        pnau: arc4.DynamicBytes,
    ) -> arc4.UInt64:
        assert self.paused.value == 0, "PAUSED"
        num_updates = self._peek_num_updates(pnau.native)
        fee = self.single_update_fee_algos.value * num_updates
        verify_fee_payment_algos(pay_txn, fee, self.aerarium.value)
        return arc4.UInt64(self._process(pnau.native))

@abimethod
def receive_price_update_usdc(
    self,
    axfer_txn: gtxn.AssetTransferTransaction,
    pnau: arc4.DynamicBytes,
) -> arc4.UInt64:
    assert self.paused.value == 0, "PAUSED"
    num_updates = self._peek_num_updates(pnau.native)
    fee = self.single_update_fee_usdc.value * num_updates
    verify_fee_payment_usdc(axfer_txn, fee, self.aerarium.value)
    return arc4.UInt64(self._process(pnau.native))

# ----- reads

@abimethod(readonly=True)
def get_price_unsafe(self, price_id: arc4.StaticArray[arc4.Byte, 32]) -> PriceStruct:
    return self.prices[price_id.bytes]

@abimethod(readonly=True)
def get_price_no_older_than(
    self,
    price_id: arc4.StaticArray[arc4.Byte, 32],
    max_age: arc4.UInt64,
) -> PriceStruct:
    p = self.prices[price_id.bytes]
    assert Global.latest_timestamp - p.publish_time.native <= max_age.native, "S
    return p

@abimethod(readonly=True)
def get_update_fee_algos(self, num_updates: arc4.UInt64) -> arc4.UInt64:
    return arc4.UInt64(self.single_update_fee_algos.value * num_updates.native)

@abimethod(readonly=True)
def get_update_fee_usdc(self, num_updates: arc4.UInt64) -> arc4.UInt64:
    return arc4.UInt64(self.single_update_fee_usdc.value * num_updates.native)

```



```

# ----- internals

@subroutine
def _peek_num_updates(self, pnau: Bytes) -> UInt64:
    vaa_start = parse_pnau_header(pnau)
    vaa_size = read_uint16_be(pnau, vaa_start)
    return op.getbyte(pnau, vaa_start + 2 + vaa_size)

@subroutine
def _process(self, pnau: Bytes) -> UInt64:
    # 1. Replay protection at parser layer.
    vaa_hash = op.sha512_256(pnau)
    assert vaa_hash not in self.seen_vaa, "VAA_REPLAY"
    self.seen_vaa[vaa_hash] = VaaReplayMarker(arc4.UInt64(Global.latest_timestamp))

    # 2. Walk PNAU header → extract VAA.
    vaa_start = parse_pnau_header(pnau)
    vaa_size = read_uint16_be(pnau, vaa_start)
    vaa = op.substring(pnau, vaa_start + 2, vaa_start + 2 + vaa_size)

    # 3. Delegate guardian-signature verification to Wormhole Core.
    payload = verify_vaa_and_return_payload(vaa, self.wormhole_core_app_id.value)

    # 4. Re-parse emitter from verified VAA bytes (defensive).
    emitter_chain = read_uint16_be(vaa, UInt64(8))
    emitter_addr = op.substring(vaa, UInt64(10), UInt64(42))
    assert emitter_chain == PYTHNET_CHAIN_ID, "EMITTER_CHAIN"
    assert emitter_addr == PYTH_EMITTER, "EMITTER_ADDR"

    # 5. AUWV root.
    root = extract_merkle_root_hash(payload)

    # 6. Iterate updates and verify each Merkle path.
    cursor = vaa_start + 2 + vaa_size
    num_updates = op.getbyte(pnau, cursor)
    cursor += 1
    applied = UInt64(0)

    i = UInt64(0)
    while i < num_updates:
        message_size = read_uint16_be(pnau, cursor)
        cursor += 2
        leaf = op.substring(pnau, cursor, cursor + message_size)

```

```

        cursor += message_size
        proof_size = op.getbyte(pnau, cursor)
        cursor += 1
        proof = op.substring(pnau, cursor, cursor + (proof_size * 20))
        cursor += proof_size * 20

        assert verify_merkle_proof(leaf, proof, proof_size, root), "PROOF"
        self._store_price(leaf)
        applied += 1
        i += 1

```

```

    return applied

```

```

@subroutine

```

```

def _store_price(self, leaf: Bytes) -> None:
    assert op.getbyte(leaf, 0) == 0, "MSG_TYPE_PRICE_FEED"
    price_id = op.substring(leaf, 1, 33)

    price_bytes = op.substring(leaf, 33, 41)
    price_neg = op.getbit(price_bytes, 0)
    price_mag = op.btoi(price_bytes)
    conf = read_uint64_be(leaf, UInt64(41))
    expo_bytes = op.substring(leaf, 49, 53)
    expo_neg = op.getbit(expo_bytes, 0)
    publish_time = read_uint64_be(leaf, UInt64(53))
    prev_publish_time = read_uint64_be(leaf, UInt64(61))
    ema_bytes = op.substring(leaf, 69, 77)
    ema_neg = op.getbit(ema_bytes, 0)
    ema_conf = read_uint64_be(leaf, UInt64(77))

    # Deviation envelope.
    if price_id in self.prices:
        prev = self.prices[price_id]
        self._check_deviation(prev.price.native, price_mag)

    self.prices[price_id] = PriceStruct(
        price=arc4.UInt64(price_mag),
        price_negative=arc4.Bool(price_neg != 0),
        conf=arc4.UInt64(conf),
        expo=arc4.UInt64(op.btoi(b"\x00\x00\x00\x00" + expo_bytes)),
        expo_negative=arc4.Bool(expo_neg != 0),
        publish_time=arc4.UInt64(publish_time),
        prev_publish_time=arc4.UInt64(prev_publish_time),
        ema_price=arc4.UInt64(op.btoi(ema_bytes)),

```

```

        ema_price_negative=arc4.Bool(ema_neg != 0),
        ema_conf=arc4.Uint64(ema_conf),
    )

@subroutine
def _check_deviation(self, old: Uint64, new: Uint64) -> None:
    if old == 0 or self.max_deviation_bps.value == 0:
        return
    diff = (new - old) if new > old else (old - new)
    assert diff * Uint64(10_000) <= self.max_deviation_bps.value * old, "DEVIATI

```

Details — Reference Price Feed Catalogue at Launch

The Pyth Stable-channel feed IDs the SDK ships with at v1.0, drawn from

<https://docs.pyth.network/price-feeds/price-feeds> and verified against

https://hermes.pyth.network/v2/price_feeds?asset_type=crypto:

Symbol	Price Feed ID (32 bytes, hex)
BTC/USD	0xe62df6c8b4a85fe1a67db44dc12de5db330f7ac66b72dc658afedf0f4a415b43 GitHub +3
ETH/USD	0xff61491a931112ddf1bd8147cd1b641375f79f5825126d665480874634fd0ace Base +3
SOL/USD	0xef0d8b6fda2ceba41da15d4095d1da392a0d2f8ed0c6c7bc0f4cfac8c280b56d GitHub Medium
AVAX/USD	0x93da3352f9f1d105fdfe4971cfa80e9dd777bfc5d0f683ebb6e1294b92137bb7 Medium
ALGO/USD	0xfa17ceaf30d19ba51112fdcc750cc83454776f47fb0112e4af07f15f4bb1ebc0 (verify against https://www.pyth.network/price-feeds/crypto-algo-usd before mainnet)
USDC/USD	0xeaa020c61cc479712813461ce153894a96a6c00b21ed0cfc2798d1f9a9e9c94a
USDT/USD	0x2b89b9dc8fdf9f34709a5b106b472f0f39bb6ca9ce04b0fd7f2e971688e2e53b
MATIC/USD	0x5de33a9112c2b700b8d30b8a3402c103578ccfa2765696471cc672bd5cf6ac52

The PYTHAI-ecosystem feeds **BKPY/USD** (BKPY supply 10 000) and **BKS/USD** (BANKON SATOSHI) are not currently published by Pyth. To list them, file a Pyth DAO listing proposal at <https://forum.pyth.network> under the *Permissionless Feed Listings* category, following the precedent of OP-PIP-94. The proposal must demonstrate publishers willing to commit prices, sufficient on-exchange notional liquidity, and signed letters from a BKPY market maker. Until listed, the SDK exposes a *synthetic* feed driver that reads BKPY from the BANKON DEX TWAP oracle on Algorand and BKS from the BANKON SATOSHI mint contract — clearly flagged in the SDK as `source = "synthetic"` not `source = "pyth"`.

Details — Client SDK (`sdk/pythai_algo_pyth_oracle_sdk/`)

python

```
# (c) 2026 BANKON — all rights reserved. Apache-2.0.
"""HermesClient — async fetcher for Pyth PNAU updates."""
from __future__ import annotations
from typing import Sequence
import httpx

HERMES_DEFAULT = "https://hermes.pyth.network"

class HermesClient:
    def __init__(self, base_url: str = HERMES_DEFAULT) -> None:
        self._client = httpx.AsyncClient(base_url=base_url, timeout=10.0)

    async def fetch_latest(self, price_ids: Sequence[str]) -> bytes:
        params = [("ids[]", pid) for pid in price_ids]
        params.append(("encoding", "hex"))
        r = await self._client.get("/v2/updates/price/latest", params=params)
        r.raise_for_status()
        body = r.json()
        return bytes.fromhex(body["binary"]["data"][0])

    async def aclose(self) -> None:
        await self._client.aclose()
```

python

```

# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""tx_builder.py – compose the atomic group that submits a PNAU to PYTHAI."""
from algokit_utils import (
    Account, AlgorandClient, AssetTransferParams, AppCallParams,
)

USDC_MAINNET = 31566704
AERARIUM_MAINNET = "AERARIUM_TREASURY_ADDR_PLACEHOLDER_58_CHARS"

def build_update_group_usdc(
    algorand: AlgorandClient,
    sender: Account,
    pythai_app_id: int,
    pnau: bytes,
    fee_microusdc: int,
) -> list[dict]:
    composer = algorand.new_group()
    composer.add_asset_transfer(
        AssetTransferParams(
            sender=sender.address, receiver=AERARIUM_MAINNET,
            asset_id=USDC_MAINNET, amount=fee_microusdc,
        )
    )
    composer.add_app_call(
        AppCallParams(
            sender=sender.address, app_id=pythai_app_id,
            method="receive_price_update_usdc",
            args=["axfer_txn", pnau],
            box_references=[],
        )
    )
    return composer.build()

```

The wallet-binding layer uses [@perawallet/connect](#), [@blockshake/defly-connect](#), and [@txnlab/use-wallet-react](#), with the signer interface being the [ClientAvmSigner](#) Protocol from [@x402-avm/avm](#) — ensuring zero-friction reuse of the user's existing x402-avm-bound wallet. [GitHub](#)

python

```

# (c) 2026 BANKON – all rights reserved. Apache-2.0.
"""api/mindx_pyth_api/app.py – FastAPI surface for the mindX agent runtime."""
from contextlib import asynccontextmanager
from fastapi import FastAPI
from x402.http.middleware.fastapi import x402_payment_middleware
from x402.http.types import RouteConfig
from x402.http import PaymentOption
from x402.schemas import Network

from .routes_read import router as read_router
from .routes_update import router as update_router
from .routes_governance import router as gov_router
from ..sdk import HermesClient

AVM_MAINNET: Network = "algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8="
AERARIUM = "AERARIUM_TREASURY_ADDR_PLACEHOLDER_58_CHARS"

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.hermes = HermesClient()
    yield
    await app.state.hermes.aclose()

app = FastAPI(title="mindX Pyth Oracle API", version="1.0.0", lifespan=lifespan)

metered_routes = {
    "POST /v1/pyth/update": RouteConfig(
        accepts=[PaymentOption(
            scheme="exact", pay_to=AERARIUM, price="$0.05", network=AVM_MAINNET,
        )],
        description="Submit a Pyth Hermes PNAU to PYTHAI.algo.pyth.oracle",
        mime_type="application/json",
    ),
}

app.add_middleware(x402_payment_middleware, routes=metered_routes)

app.include_router(read_router, prefix="/v1/pyth")
app.include_router(update_router, prefix="/v1/pyth")
app.include_router(gov_router, prefix="/v1/pyth/governance")

```

The MCP server (`mcp_server.py`) exposes three tools — `pyth_get_price`, `pyth_update_price`, `pyth_list_feeds` — to the BANKON agentic runtime. Each tool's input/output schemas are auto-generated from the FastAPI Pydantic models using the `fastmcp` adaptor.

Details — Foundry Tests for the EVM Consumer (DELTAVERSE)

solidity


```

// foundry/src/DeltaversePythConsumer.sol
// (c) 2026 BANKON – all rights reserved. Apache-2.0.
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.24;

import "@pythnetwork/pyth-sdk-solidity/IPyth.sol";
import "@pythnetwork/pyth-sdk-solidity/PythStructs.sol";

contract DeltaversePythConsumer {
    IPyth public immutable pyth;
    address public immutable openBdkRelayer;
    mapping(bytes32 => PythStructs.Price) public bridgedPrices;

    constructor(address _pyth, address _relayer) {
        pyth = IPyth(_pyth);
        openBdkRelayer = _relayer;
    }

    function postBridgedPrice(
        bytes32 priceId,
        PythStructs.Price calldata price,
        bytes calldata bridgeProof
    ) external {
        require(msg.sender == openBdkRelayer, "RELAYER_ONLY");
        bridgedPrices[priceId] = price;
    }

    function getBridgedPriceNoOlderThan(bytes32 priceId, uint256 maxAge)
        external view returns (PythStructs.Price memory p)
    {
        p = bridgedPrices[priceId];
        require(block.timestamp - uint256(uint64(p.publishTime)) <= maxAge, "STALE")
    }
}

```

solidity

```

// foundry/test/DeltaversePythConsumer.t.sol
pragma solidity ^0.8.24;

import "forge-std/Test.sol";
import "@pythnetwork/pyth-sdk-solidity/MockPyth.sol";
import "@pythnetwork/pyth-sdk-solidity/PythStructs.sol";
import "../src/DeltaversePythConsumer.sol";

contract DeltaversePythConsumerTest is Test {
    MockPyth pyth;
    DeltaversePythConsumer consumer;
    address relayer = address(0xBEEF);
    bytes32 constant BTC = 0xe62df6c8b4a85fe1a67db44dc12de5db330f7ac66b72dc658afedf0

    function setUp() public {
        pyth = new MockPyth(60, 1);
        consumer = new DeltaversePythConsumer(address(pyth), relayer);
    }

    function test_bridged_price_round_trips() public {
        PythStructs.Price memory p = PythStructs.Price({
            price: 105_000_000000000, conf: 5_000000000,
            expo: -8, publishTime: uint64(block.timestamp)
        });
        vm.prank(relayer);
        consumer.postBridgedPrice(BTC, p, "");
        PythStructs.Price memory got = consumer.getBridgedPriceNoOlderThan(BTC, 120);
        assertEq(got.price, 105_000_000000000);
    }

    function test_only_relayer_can_post() public {
        PythStructs.Price memory p;
        vm.expectRevert("RELAYER_ONLY");
        consumer.postBridgedPrice(BTC, p, "");
    }

    function test_stale_reverts() public {
        PythStructs.Price memory p = PythStructs.Price({
            price: 1, conf: 0, expo: 0,
            publishTime: uint64(block.timestamp - 3600)
        });
        vm.prank(relayer);
        consumer.postBridgedPrice(BTC, p, "");
    }
}

```

```
        vm.expectRevert("STALE");
        consumer.getBridgedPriceNoOlderThan(BTC, 60);
    }
}
```

MockPyth ships in `@pythnetwork/pyth-sdk-solidity` and is documented at https://github.com/pyth-network/pyth-crosschain/tree/main/target_chains/ethereum/sdk/solidity. [GitHub](#) [npm](#)

Details — AlgoKit Test Suite

python

```

# tests/test_pyth_oracle.py
# (c) 2026 BANKON — all rights reserved. Apache-2.0.
from algopy_testing import algopy_testing_context
from contracts.pyth_oracle import PythAlgoOracle

def test_basic_update_and_read():
    with algopy_testing_context() as ctx:
        oracle = PythAlgoOracle()
        from tests.vectors import make_pnau_btc_100k
        pnau = make_pnau_btc_100k()
        pay = ctx.any.txn.payment(
            sender=ctx.default_sender, receiver=oracle.aerarium.value, amount=1_000
        )
        n = oracle.receive_price_update_algos(pay, pnau)
        assert n.native == 1
        from sdk.pythai_algo_pyth_oracle_sdk.feeds import BTC_USD
        p = oracle.get_price_unsafe(BTC_USD)
        assert p.price.native == 100_000 * 10**8

def test_replay_rejected():
    with algopy_testing_context() as ctx:
        oracle = PythAlgoOracle()
        from tests.vectors import make_pnau_btc_100k
        pnau = make_pnau_btc_100k()
        pay = ctx.any.txn.payment(
            sender=ctx.default_sender, receiver=oracle.aerarium.value, amount=1_000
        )
        oracle.receive_price_update_algos(pay, pnau)
        import pytest
        with pytest.raises(AssertionError, match="VAA_REPLAY"):
            oracle.receive_price_update_algos(pay, pnau)

```

The fuzz suite (`test_fuzz_payload.py`) uses Hypothesis to randomly perturb PNAU bytes and asserts that the parser either accepts the payload (and the resulting Merkle verification succeeds) or aborts cleanly — no panics, no underflow, no silent corruption. The replay-and-fork suite (`test_fork_replay_hermes.py`) loads JSON-encoded captures of real mainnet Hermes responses for BTC/USD, ETH/USD and ALGO/USD and plays them through a LocalNet with a mocked `wormhole_core` that signs anything (sufficient to exercise PNAU/AUWV/Merkle paths end-to-end). Fuzz-testing Algopy contracts is documented at <https://algorandfoundation.github.io/algorand-python-testing/>. `Algorand`

Details — Deployment Scripts

python

```
# deploy/deploy_mainnet.py - invoked once, by the Senatus 5-of-7 multisig.
# (c) 2026 BANKON - all rights reserved. Apache-2.0.
from algokit_utils import AlgorandClient, get_account
from algopy import arc4
from contracts.pyth_oracle import PythAlgoOracle
import os

WORMHOLE_CORE_MAINNET = 842_125_965
AERARIUM_MAINNET = os.environ["AERARIUM_ADDR"]
SENATUS_MAINNET = os.environ["SENATUS_MULTISIG_ADDR"]

def main():
    algorand = AlgorandClient.from_environment()
    deployer = get_account("DEPLOYER", algorand)
    client = algorand.client.from_algorand_python(PythAlgoOracle, deployer)
    app = client.create()
    client.send.set_aerarium(arc4.Address(AERARIUM_MAINNET))
    client.send.rotate_senatus(arc4.Address(SENATUS_MAINNET))
    print(f"PYTHAI.algo.pyth_oracle app_id = {app.app_id}")
    print(f"App addr = {app.app_address}")
    print(f"Wormhole core (mainnet) = {WORMHOLE_CORE_MAINNET}")

if __name__ == "__main__":
    main()
```

The CAIP-2 row added to agentiplace.pythai.net/allchain.html is:

html

```
<tr data-caip2="algorand:wGHE2Pwddvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=">
  <td>Algorand Mainnet</td>
  <td>algorand:wGHE2Pwddvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=</td>
  <td>PYTHAI.algo.pyth.oracle</td>
  <td><code id="pythai-app-id">{{INJECTED_AT_DEPLOY}}</code></td>
  <td>USDC ASA 31566704 · Wormhole Core 842125965</td>
</tr>
```

NFD (`https://app.nf.domains`) registration of `PYTHAI.algo.pyth.oracle` under the `PYTHAI` namespace ties the app ID to a human-readable name and survives migration; the NFD payment field `app_id_PYTHAI.algo.pyth.oracle` carries the deployed integer.

Details — Minimal Reactive UI

tsx

```
// ui/PythPriceTicker.tsx - single-file, Tailwind, use-wallet, no extras.
// (c) 2026 BANKON - all rights reserved. Apache-2.0.
import { useEffect, useState } from "react";

const FEEDS = [
  ["BTC/USD", "e62df6c8b4a85fe1a67db44dc12de5db330f7ac66b72dc658afedf0f4a415b43"],
  ["ETH/USD", "ff61491a931112ddf1bd8147cd1b641375f79f5825126d665480874634fd0ace"],
  ["SOL/USD", "ef0d8b6fda2ceba41da15d4095d1da392a0d2f8ed0c6c7bc0f4cfac8c280b56d"],
  ["AVAX/USD", "93da3352f9f1d105fdfe4971cfa80e9dd777bfc5d0f683ebb6e1294b92137bb7"],
  ["ALGO/USD", "fa17ceaf30d19ba51112fdcc750cc83454776f47fb0112e4af07f15f4bb1ebc0"],
  ["USDC/USD", "eaa020c61cc479712813461ce153894a96a6c00b21ed0cfc2798d1f9a9e9c94a"],
  ["USDT/USD", "2b89b9dc8fdf9f34709a5b106b472f0f39bb6ca9ce04b0fd7f2e971688e2e53b"],
  ["MATIC/USD", "5de33a9112c2b700b8d30b8a3402c103578ccfa2765696471cc672bd5cf6ac52"],
] as const;

export default function PythPriceTicker() {
  const [prices, setPrices] = useState<Record<string, { p: number; ts: number }>>({});
  useEffect(() => {
    const tick = async () => {
      const next: typeof prices = {};
      await Promise.all(FEEDS.map(async ([sym, id]) => {
        const j = await fetch(`/v1/pyth/price/${id}?max_age=60`).then(r => r.json());
        next[sym] = { p: j.price * Math.pow(10, j.expo), ts: j.publish_time };
      }));
      setPrices(next);
    };
    tick();
    const h = setInterval(tick, 5000);
    return () => clearInterval(h);
  }, []);
  return (
    <div className="grid grid-cols-2 md:grid-cols-4 gap-3 p-4 font-mono">
      {FEEDS.map(([sym]) => (
        <div key={sym} className="rounded-xl border p-3 bg-neutral-950 text-emerald-400">
          <div className="text-xs opacity-70">{sym}</div>
          <div className="text-2xl">${prices[sym]?.p.toFixed(2)} ?? "-"</div>
        </div>
      ))}
    </div>
  );
}
```

`ui/GovernanceProposal.tsx` follows the same single-file philosophy: a Tessera holder connects via `useWallet`, fills a form (`set_pause`, `set_default_max_age`, `set_max_deviation_bps`, `rotate_senatus`), the form POSTs to `/v1/pyth/governance/propose`, which creates a Senatus 5-of-7 partial-multisig blob; signers add signatures over the next 72 hours, the server submits when threshold is met.

Details — CONCLAVE Integration (Treasurer + Risk Counsellors)

python

```
# agent/conclave_treasurer.py - Treasurer Counsellor uses pyth_get_price.
# (c) 2026 BANKON - all rights reserved. Apache-2.0.
from mindx.cognition import Counsellor

class TreasurerCounsellor(Counsellor):
    name = "Treasurer"

    async def value_portfolio(self, holdings: dict[str, float]) -> float:
        total = 0.0
        for sym, amt in holdings.items():
            p = await self.tool(
                "pyth_get_price", price_id=self.feed_id(sym), max_age_sec=120
            )
            total += amt * p["price"] * (10 ** p["expo"])
        return total

class RiskCounsellor(Counsellor):
    name = "Risk"

    async def watch_deviation(self) -> None:
        async for tick in self.hermes_stream():
            if self.deviation_60s(tick) > 0.05:
                await self.request_senatus_pause()
```

Recommendations (decision-ready, staged)

Stage 1 — Build & ship a TestNet beta (Weeks 1-3). Gregory, your highest-leverage move is to scaffold the repo above and deploy `PYTHAI.algo.pyth.oracle` against the Algorand TestNet

Wormhole Core (App ID **86525623**, per

<https://wormhole.com/docs/products/reference/contract-addresses/>) with Pyth Beta-channel feed IDs. You have everything you need: the Wormhole Algorand integration is the load-bearing primitive and it is mature. The PNAU/AUWV format is fully documented through the Stacks bridge mirror. Allocate Week 1 to the parser + Merkle verifier (pure Algopy, easy to fuzz offline); Week 2 to the Wormhole inner-app-call delegation and box storage; Week 3 to the FastAPI / x402-avm fee plumbing and a 30-minute end-to-end demo updating BTC/USD live.

Stage 2 — Audit and mainnet deployment (Weeks 4-7). Engage an Algopy-fluent auditor. The strongest Algorand security-engineering signal in 2025 was **Adevar's** October 2025 audit of the Folks Finance Algorand Wormhole NTT contracts (<https://github.com/Folks-Finance/audits/blob/main/Adevar%20-%20Algorand%20Wormhole%20NTT%20-%20October%202025.pdf>) — that team has the exact context (AVM + Wormhole VAA verification flow) that `PYTHAI.algo.pyth.oracle` requires. Trail of Bits, Vantage Point, and CertiK (the Folks Finance V2 auditors, per the Folks Finance FAQ and <https://github.com/Folks-Finance/audits>) are credible alternatives; Runtime Verification audited Folks Finance V1 design only. Focus the engagement on the Merkle verifier, the PNAU parser's bounds checks, the deviation-envelope arithmetic (overflow risk on `diff * 10_000`), and the replay-marker box scheme. Budget around \$40–60k. Concurrently file the BKPY/USD and BKS/USD Pyth DAO listing proposal — it has a long lead time and is independent of your code. Mainnet deploy via Senatus 5-of-7 once audit is clean; pin the resulting app ID into agenticplace.pythai.net/allchain.html and the NFD record. `Folks + 2`

Stage 3 — Negotiate canonical adoption with Pyth (Weeks 8-12). Once `PYTHAI.algo.pyth.oracle` is operating on mainnet with non-trivial daily update volume, open a Pyth DAO proposal in the *Onchain Programs* category (precedent: OP-PIP-94, <https://forum.pyth.network/t/passed-op-pip-94-q1-2026-entropy-protocol-fee-implementation/2347>) requesting that Pyth adopt your contract as the **canonical Algorand client**, add it to <https://docs.pyth.network/price-feeds/core/contract-addresses>, and accept upstream maintenance. The leverage you have: Pyth has been carrying the Algorand "up next" promise unfulfilled since 2022, C3 Exchange is a multi-million-tx-per-day Algorand consumer that would benefit from a canonical client, and you arrive with a working production deployment, audit reports, and CI.

Thresholds that would change these recommendations. (a) If Pyth ships its own Algorand client before Stage 2 mainnet, kill PYTHAI and integrate against the official one; the lift to switch is around two days because the consumer API surface (`get_price_no_older_than`, `get_update_fee`, `update_price_feeds`) is the universal Pyth shape. (b) If the Algorand AVM cost of a single Merkle proof + box write exceeds 30 inner-txns and 4 ms of TEAL, factor out the Merkle verifier into a stateless `LogicSig` to amortise cost — Algopy supports `subroutine`-emitted lsigs. (c) If x402-avm v3 ships breaking changes to `ALGORAND_MAINNET_CAIP2` semantics, pin to `x402-avm==2.7.x` until the agentic stack catches up. `Etherscan` `GitHub`

Caveats

1. **The exact Wormhole-Core call signature on Algorand (`verifyVAA`)** that returns a payload via Log is paraphrased here from the `wormhole_core.py` source; in production the call pattern requires a parallel `VaaVerify` stateless lsig group covering the guardian signatures (the `tmpl_sig` 2048-bit bitfield mechanism described at <https://github.com/wormhole-foundation/wormhole/tree/main/algorand>). The pseudo-code above models the post-`verifyVAA` payload retrieval; the full pre-grouped lsig submission is well-documented in `wormhole_core.py` but adds roughly three extra transactions to every update group. Budget for that in your fee calculations. [GitHub](#) [GitHub](#)
2. **The `BKPY/USD` and `BKS/USD` feed IDs do not exist on Pyth today.** Until and unless the Pyth DAO accepts the listing proposal, the SDK exposes them only as `source="synthetic"` driven by BANKON-internal TWAP oracles. Do not let downstream consumers treat them as Pyth-attested.
3. **The `ALGO/USD`, `USDC/USD`, `USDT/USD`, `AVAX/USD`, and `MATIC/USD` price feed IDs in the launch catalogue** above are sourced from cross-referenced docs and the per-asset Pyth feed pages but were not all independently re-quoted from `pyth-crosschain` source within the research budget. Before mainnet deploy, run `curl 'https://hermes.pyth.network/v2/price_feeds?asset_type=crypto' | jq '[] | select(.attributes.symbol=="Crypto.ALGO/USD")'` for each and persist the authoritative IDs into `sdk/feeds.py` as the single source of truth. `BTC/USD`, `ETH/USD`, and `SOL/USD` are confirmed verbatim from primary Pyth-controlled sources.
4. **The Pyth emitter address** `0xe101faedac5851e32b9b23b5f9411a8c2bac4aae3ed4dd7b811dd1a72ea4aa71` is derived from `pythnet_sdk`'s `ACCUMULATOR_EMITTER_ADDRESS` (the PDA of `[b"emitter"]`) under the Pythnet accumulator program). Confirm by inspecting `dataSourceEmitterAddresses` on a deployed `PythUpgradable` contract before hard-coding into mainnet bytecode.
5. **C3 Exchange's prior Pyth-on-Algorand usage** does not constitute a canonical receiver. Their architecture pushes Pyth prices from their off-chain risk engine into a private health-calculator contract — this is a private-key-signed shortcut, not a Wormhole-attested VAA verification path. Your work is not redundant with C3's; you are filling a real public-infrastructure gap.
6. **Algopy/PuyaPy is still moving fast.** `algorand-python 3.5.0` (released April 8, 2026 with PyPI classifier `Development Status :: 5 - Production/Stable`), per <https://pypi.org/project/algorand-python/>) is sufficient for everything in this design (`BoxMap` with ARC4 structs, `op.keccak256`, inner app calls). However the `op.substring` opcode-cost model has changed multiple times during the v3.x line. Budget time to re-benchmark TEAL opcode cost on the latest LocalNet image before mainnet deployment.

7. **Pyth chose not to ship Algorand for roughly three years** despite explicit roadmap promise. The likeliest reason is the AVM's lack of native secp256k1 ecrecover, which forces Wormhole guardian signature verification to be done via stateless-lsig groups rather than a single opcode — i.e., it is genuinely more painful than other chains. That is precisely why `PYTHAI.algo.pyth.oracle` is the right thing to build *and* is unlikely to be obsoleted by an official Pyth shipping later in 2026.